RAMCO INSTITUTE OF TECHNOLOGY

Rajapalayam, Tamilnadu, India

DEPT. OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR – 2019-2020 (EVEN)

Course code & Title: CS6801-Multi-core Architecture and Programming

Semester–VII

Date:03.02.2020

Time:1.25pm to 02.15pm

Topic: OpenMP Execution Model

HANDS-ON CLASS ACTIVE LEARNING

Mr.M.GomathyNayagam., M.E., Assistant Professor, (SG)

1. Objectives:

- Make the students to execute simple programs using openMP
- To enable the students to obtain the idea of developing parallel applications in C/C++ with openMP

2. OpenMP.

An Application Program Interface (API) that may be used to explicitly direct multi-threaded, shared memory parallelism. It comprised of three primary API components:Compiler Directives,Runtime Library Routines, Environment Variables. An abbreviation forOpenMP is Open Multi-Processing. OpenMP is a set of compiler directives as well as an API for programs written in C, C++, or FORTRAN that provides support for parallel programming in shared-memory environments. OpenMP identifies parallel regions as blocks of code that may run in parallel. Application developers insert compiler directives into their code at parallel regions, and these directives instruct the OpenMP run-time library to execute the region in parallel.OpenMP is not required to check for data dependencies, data conflicts, race conditions, deadlocks, or code sequences that cause a program to be classified as non-conforming and designed to handle parallel I/O. The programmer is responsible for synchronizing input and output. The main goals of OpenMP are: Standardization, lean and mean, Ease of use and portability.

In this hands-on session, I trained the student in our local telnet server where it already contains OpenMP API

The following command is used to access our telnet server.

telnet 172.16.0.94

Each student used their own login id to use telnet server.

2.1. OpenMPRelease History:

- OpenMP continues to evolve - new constructs and features are added with each release.
- Initially, the API specifications were released separately for C and Fortran.
- Since 2005, they have been released together.

Date	Version
Oct 1997	Fortran 1.0
Oct 1998	C/C++ 1.0
Nov 1999	Fortran 1.1
Nov 2000	Fortran 2.0
Mar 2002	C/C++ 2.0
May 2005	OpenMP 2.5
May 2008	OpenMP 3.0
Jul 2011	OpenMP 3.1
Jul 2013	OpenMP 4.0
Nov 2015	OpenMP 4.5

3. Manual

3.1 OpenMP Programming Model.

OpenMP uses Fork-join model for parallel execution shown in fig.1. All OpenMP programs begins as a single process which is called as Master thread. Master thread spawns a team of threads as needed. The master thread executes sequentially until the first parallel region construct is encountered. Fork means the master thread creates a team of parallel threads. The statements in the program that are enclosed by the parallel region construct are then executed in parallel among the various team threads. Join means that when the team threads complete the statements in the parallel region construct, they synchronize and terminate, leaving only the master thread.The number of parallel regions and the threads that comprise them are arbitrary.

Parallelism added incrementally until performance goals are met: i.e. the sequential program evolves into a parallel program.

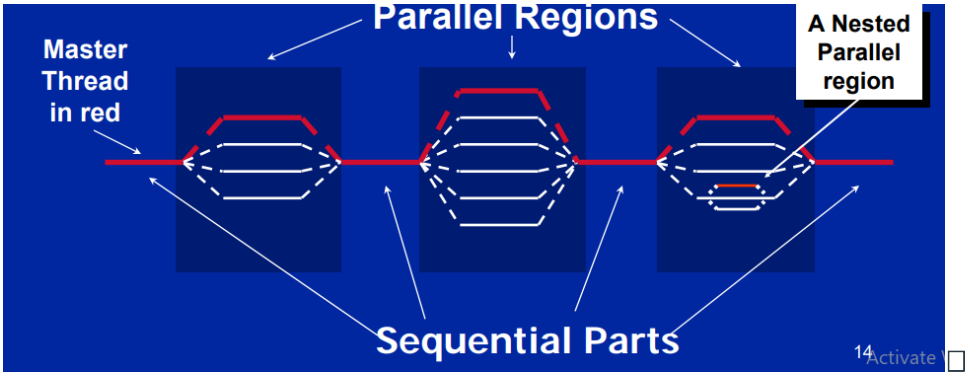

Fig.1. Fork-Join Parllelsim

3.2. OpenMP Stack:

The following fig.2. shows the OpneMP API stacks. The programmer can uses either OpenMP compiler directive or library or environment variables which are available in programming layer of the stack to develop their own application on the user layer. These directives interact with OpenMP runtime library which is running over the OS in system layer to create a multi-thread for parallelizing the application. OpenMP directives interact shared memory address space through system layer.

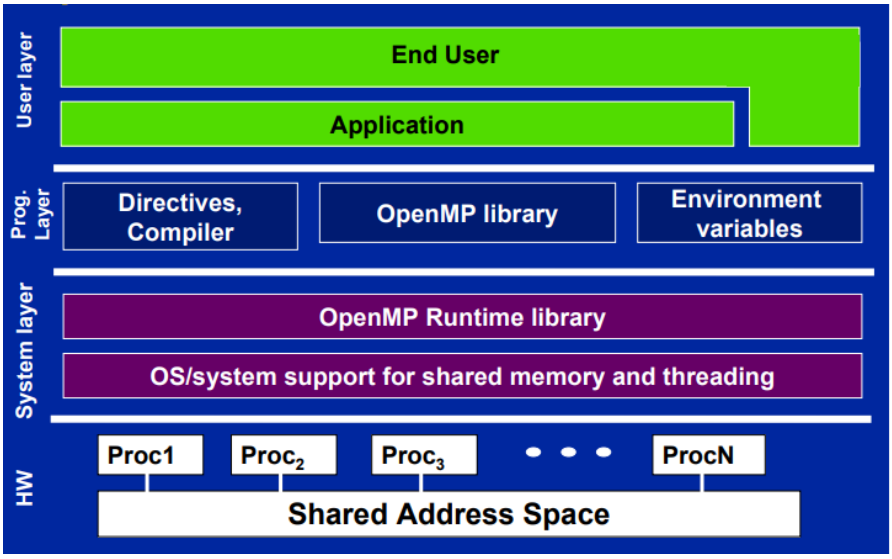

Fig.2. OpenMP API stack

3.3. OpenMP core Syntax:

Most of the programming constructs in OpenMP are compiler directives. The syntax for omp construct is:

#pragma omp construct [clause [clause]…]

An example foromp construct is :

#pragma omp parallel num_threads(4)

In order to access the compiler directives or library or environment variable openmp, the programmer must include omp.h header file in their application..

#include<omp.h>

Most of the OpenMP constructs are applied to a "Structured block of code" where a block of one or more statements with one point of entry at the top and one point of exit at the bottom.

3.4. OpenMP installation

After the evaluation of gcc version 4 in linux, there is no need to install openmp API separately. Omp is built-in to gcc compiler. To check openmp version, use the following command in terminal.

$echo |cpp -fopenmp -dM |grep -i open

It will show the result as

#define _OPENMP 200805

Then, it will be ok for creating shared memory application by the programmer.

3.5. Compile and Run Sample Program:

- Before compile and run our program in openmp, check the information about our processor by using the following commands

$lscpu

- According to the capability of our telnet server, it will shows the result as follows:

```
Architecture:          i686
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                8
On-line CPU(s) list:   0-7
Thread(s) per core:    1
Core(s) per socket:    4
Socket(s):             2
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 45
Stepping:              7
CPU MHz:               1200.000
BogoMIPS:              4799.35
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              10240K
```

- From the above results, we can conclude, we have 8 cpus (cores), each can run one thread at a time.
- Open one vi editor. Give the filename with .c extension and type the following code

```
#include<stdio.h>
#include<omp.h>
int main()
{
        #pragma omp parallel
        {
                printf("Welcome");
        }
        retrun 0;
}
```

- The output of the above program is as follows:

```
Welcome from thread 0
Welcome from thread 1
Welcome from thread 2
Welcome from thread 3
Welcome from thread 4
Welcome from thread 5
Welcome from thread 6
Welcome from thread 7
```

- Because of 8 cores cpu which can handle single thread at a time in server where the program has been executed, the 8 thread has been created and executed concurrently.
- But the execution order of the thread may be changed. There is no need to execute the thread in order in which it was created.

4. Frequent Asked Questions and answers.

1. What is OpenMP?

OpenMP is a specification for a set of compiler directives, library routines, and environment variables that can be used to specify high-level parallelism in Fortran and C/C++ programs

2. Why should I use OpenMP?

There are several reasons why you should use OpenMP: (a) OpenMP is the most widely standard for SMP systems,it supports 3 different languages (Fortran, C, C++), and it has been implemented by many vendors. (b) OpenMP is a relatively small and simple specification, and it supports incremental parallelism. (c) A lot of research is done on OpenMP, keeping it up to date with the latest hardware developments.

3. What languages does OpenMP support ?

OpenMP is designed for Fortran, C and C++. OpenMP can be supported by compilers that support one of Fortran 77, Fortran 90, Fortran 95, Fortran 2003, Fortran 2008, C11, C++11, and C++14, but the OpenMP specification does not introduce any constructs that require specific Fortran 90 or C++ features

4. Is OpenMP scalable ?

OpenMP can deliver scalability for applications using shared-memory parallel programming. Significant effort was spent to ensure that OpenMP can be used for scalable applications. However, ultimately, scalability is a property of the application and the algorithms used. The parallel programming language can only support the scalability by providing constructs that simplify the specification of the parallelism and can be implemented with low overhead by compiler vendors. OpenMP certainly delivers these kinds of constructs.

5. Can I use OpenMP to program accelerators ?

OpenMP provides mechanisms to describe regions of code where data and/or computation should be moved to another computing device. OpenMP supports a broad array of accelerators, and plans to continue adding new features based on user feedback.

6. How does OpenMP compare with Pthreads ?

Pthreads have never been targeted toward the technical/HPC market. This is reflected in the minimal Fortran support, and its lack of support for data parallelism. Even for C applications, pthreads requires programming at a level lower than most technical developers would prefer.

5. Reflection Report

- All the students worked and execute simple openMP program comfortably because its simplicity.
- Students get clear fundamental knowledge of multi-core architecture and get the hands-on about the creation of thread by multi-core processor.
- Students worked with joyfully because of visualization blow in theirmind.
- I assisted to slow learners to execute the program in their systems.
- I faced a lot of issues when students enter in to the telnet server log in for entering into openmp environment.
- I conducted this hands-on sessions for executing different application program (convert sequential program to parallel) of opemMP. I felt that few students struggling to convert sequential application into parallel applications.

- ❖ All the students are given good feedback and some of the students preferred their final year project in this field

6. Conclusion

OpenMP API provides a portable, scalable model for developers of shared memory parallel application. It also supports C/C++ and Fortran on wide variety of architectures. This hands-on session helps the students to convert serial application to parallel application which can run on shared memory environment.

References:

1. https://www.openmp.org
2. *www.doc.ic.ac.uk / ~wjk / openmp-slides*
3. *www.geeksforgeeks.org / openmp-hello-world-program*
4. *www.tutorialspoint.com › what-is-openmp*
5. *www.bowdoin.edu › teaching/ cs3225-GIS › fall16 › Lectures › ope..*
6. *www.cse.iitm.ac.in › ~rupesh / teaching/› hpc/› jun16/› 4-openmp*

Prepared by
M.Gomathy Nayagam, AP(SG)/CSE

Verified by
HOD/CSE